

Visual-Inertial Odometry Algorithm for GPS-denied Localization of an FPV Drone

Sahil Kale, Dhruv Upadhayay, Akshat Doctor, Owen Weiss, and Aaron Spoelstra
University of Waterloo, Waterloo, Ontario, Canada

This paper describes the implementation of a visual-inertial odometry localization algorithm on an FPV drone. The UZH-FPV Drone Racing Dataset ([1]) was used to tune and evaluate the performance of the algorithm. A cascaded filtering approach — combining a vision-based point cloud matching algorithm, a Madgwick filter, and a Kalman filter — was employed to estimate the drone’s pose and position in the world frame with reasonable accuracy. The results show the drone was able to localize itself in the world frame with a root mean square error (RMSE) of [5.02, 3.17, 4.30] in position and ≤ 0.05 rad in orientation. The implemented VIO algorithm only requires IMU and camera data, making it suitable for use in GPS-denied or off-world environments.

I. INTRODUCTION

Visual-inertial odometry (VIO) is the process of localization using camera and IMU data. It is commonly used in autonomous applications where other methods such as LIDAR or GPS are infeasible. These can include operating in GPS-denied environments such as in defense or mining applications, as well as scenarios where weight is a significant factor.

VIO has been successfully implemented in many applications, and a large amount of literature exists on the implementation and validation of these systems. A notable example of VIO use is the NASA Mars Helicopter, which used VIO in combination with an altimeter to enable autonomous flight on a highly mass-optimized platform [2]. To reduce the computational load compared to commonly implemented SLAM algorithms, the Mars helicopter used velocimetry, using the vision system to determine relative motions as opposed to absolute position. Specifically, the algorithm used is a derivative of an Extended Kalman Filter (EKF) known as MAVen (minimally augmented state algorithm for vision-based navigation).

II. BACKGROUND INFORMATION

A. Kalman Filter Theory

A Kalman Filter is a type of algorithm which predicts the states of a system using their dynamics, and correcting the prediction with feedback from an external sensor. The filter is used to fuse the data coming from the IMU and the data from the camera. In the case of the project, the Kalman filter was used to predict the position of the drone relative to the world frame. The states of the Kalman Filter were thus

$$[t_x, t_y, t_z, v_x, v_y, v_z] \quad (1)$$

which represent the translation and velocity of the drone in world frame. The inputs of the system were the IMU acceleration data and the camera distance output, while the output of the system was the updated position of the drone.

The filter uses various noise and covariance matrices to add uncertainty to the model. P, Q, and R are the main

matrices. The state covariance matrix (P), is responsible for representing the uncertainty in the system states. It essentially represents the trust in the state variables versus the measured values. The Q and R covariance matrices represent two different noises. Q represents the process noise matrix, which models the uncertainty in the system dynamics. R represents the measurement noise covariance, which models how noisy the sensor measurement is.

Together, these matrices, along with the calculated outputs from the predict and measurement stages are used to calculate the Kalman gain. The gain is a measure of the difference between the actual and predicted measurements, and is modulated accordingly using the difference between the predicted and measured states. This is then used to update the state of the system.

B. Madgwick Filter Background

The Madgwick filter is an orientation filter that uses quaternions to describe the orientation from a fixed body frame to a moving inertial frame [3]. As a result of the algorithms using quaternions to represent the pose of the drone, it does not suffer from known drawbacks of using euler angles as a pose representation (such as gimbal lock).

1) Gyroscopic Integration Step

The first step of a Madgwick filter is to integrate the gyroscopic measurements (angular rate about the body frame axes, represented by $[\omega_x \ \omega_y \ \omega_z]$) to get a new estimation of the inertial frame’s orientation, shown in Equation 2.

$$q_k = q_{k-1} + \frac{q_{k-1} \otimes [0 \ \omega_x \ \omega_y \ \omega_z] \Delta t}{2} \quad (2)$$

2) Accelerometer Correction Step

The second step of a Madgwick filter is to correct the orientation estimate using the accelerometer measurements. The accelerometer measurements are represented in the body frame as $[a_x \ a_y \ a_z]$. The goal of the correction step is to align the accelerometer measurements in the world frame with the gravity vector in the world frame, represented as $[0 \ 0 \ g]$. The gravity vector is normalized to $[0 \ 0 \ 1]$ to simplify the equations. Note the gravity vector pointing “up” is due

to how the sensor frame measurements are oriented. From this, an optimization problem can be formed to minimize the difference between the two vectors, shown in Equations 3 and 4.

$$f_g(q, s_a) = \begin{bmatrix} 2(q_x q_z - q_w q_y) - a_x \\ 2(q_w q_x + q_y q_z) - a_y \\ 2\left(\frac{1}{2} - q_x^2 - q_y^2\right) - a_z \end{bmatrix} \quad (3)$$

$$J_g(q) = \begin{bmatrix} -2q_y & 2q_z & -2q_w & 2q_x \\ 2q_x & 2q_w & 2q_z & 2q_y \\ 0 & -4q_x & -4q_y & 0 \end{bmatrix} \quad (4)$$

The gradient of the optimization function is calculated with the following Equation 5:

$$\nabla f_g(q, s_a) = J_g(q)^T f_g(q, s_a) \quad (5)$$

A correctional step is then performed using the gradient of the optimization function, as shown in Equation 6. Note the step size μ is a tunable parameter that can be adjusted to change the aggressiveness of the correction step.

$$q_k = q_k - \mu \frac{\nabla f_g(q, s_a)}{\|\nabla f_g(q, s_a)\|} \Delta t \quad (6)$$

III. METHODS AND MATERIALS

A. Data Collection

The data was collected using the dataset from the paper "Are We Ready for Autonomous Drone Racing? The UZH-FPV Drone Racing Dataset" [1]. The paper contains 27 different flight paths with camera data, inertial and ground truth positioning to set up robust pose and position tracking algorithms.

The data was collected by flying a small FPV drone around various different indoor and outdoor paths, recording the pertinent data mentioned above. The FPV drone used by the team contained measurements from a stereo camera, an IMU and a reflector (for tracking ground truth). Figure 1 shows the setup of the physical drone.

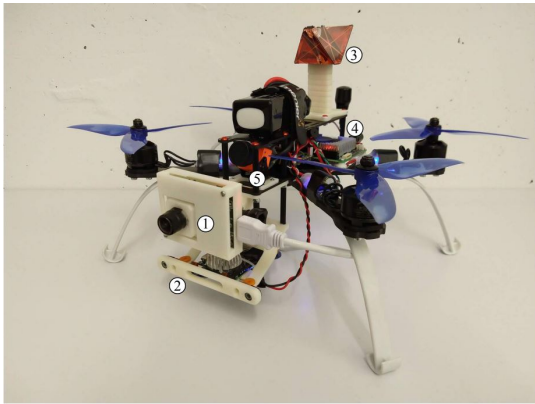


Fig. 1. Drone Measurement Setup. : 1 mDAVIS 2 Snapdragon Flight with fisheye stereo cameras 3 Omnidirectional reflector that provides the ground truth position. 4 Up Board used for recording the DAVIS data. 5 FPV camera used by the human pilot [1]

For visual-inertial odometry (VIO), only the camera and IMU are essential; the reflector is used solely for obtaining ground truth. A key part of the implementation involves transforming between coordinate frames—specifically from the camera and IMU frames to the world frame. Since the IMU and drone have different axes, and the drone's orientation changes as it moves, a rotation is required to align the IMU frame with the drone frame. The Madgwick filter was used to estimate the drone's pose in the world frame. Additional transformations were applied to relate the camera frame to both the drone and world frames. Figure 2 illustrates the different reference axes of the onboard sensors

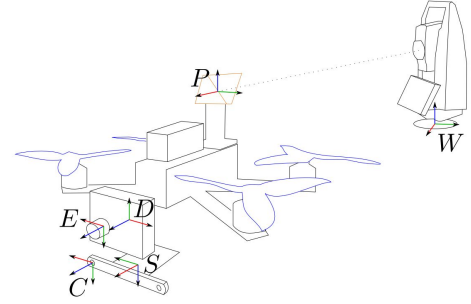


Fig. 2. Drone Axis: C: Camera Frame, S: IMU Frame, P: Reflector Frame/Drone Frame, W: World Frame [1]

B. Vision Relative Odometry Implementation

Visual odometry involves extracting information from a series of images in order to estimate the pose and position of a camera. The dataset provides 30 frames per second stereo images allowing for 3D localization. OpenCV was used for the image extraction and processing [4]. The overall process is outlined in Figure 3.

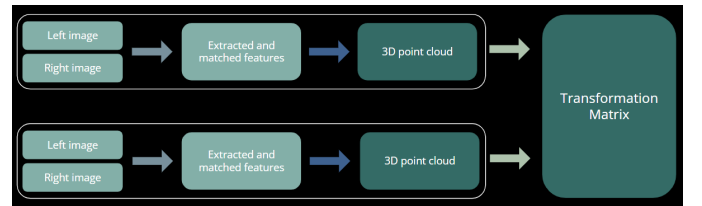


Fig. 3. High-level process for relative visual odometry.

Image features were first extracted from left and right pairs of images at a given time step. The Scale-Invariant Feature Transform (SIFT) feature extraction algorithm was used. The features of the two images were then matched to ensure the same points in both are compared. The Fast Library for Approximate Nearest Neighbors (FLANN) matcher was used to create feature matches between the two images. Random sample consensus (RANSAC) was then used to filter these matches, only preserving the highest quality matches. These steps were repeated for the previous time step, and the filtering was again applied, this time between the left images of the previous and current time step. This ensured that the two point clouds generated contain the same points such that

the transformation between them can be found.

With the same features identified in both the left and right images, a 3D point cloud was generated by using triangulation to find the location of the points in 3D space. The image was first undistorted to match a pinhole camera model. This was done by using the intrinsic transformation matrix provided in the calibration data. The point clouds at sequential time steps were then generated using triangulation, producing two point clouds with ideally the same points. This process is visualized in Appendix A.

To find the transformation between these two point clouds, outlier points, such as points behind the camera, were removed. A custom iterative method was then used to find the transformation. A first guess transformation was applied to the previous time step's point cloud known as the source. The distances between the points in the source were now compared to the current time step's (destination) point cloud. Weights for each point were generated that are inversely proportional to the distances. The centroid of the point cloud was then found using the weights, and the centroids were aligned. The weighted covariance was found and PCA was used to obtain the principal components of the covariance. These principal axes were then aligned, and the rotation required to do this is saved as the rotation matrix. This rotation could then be applied to the source point cloud, and the translation between them was found through simple subtraction. This transformation was then fed back to the start of the process, and was repeated until the transformation matrix converged. A graphic visualizing alignment is shown in Appendix A.

The system returns a relative transformation vector, t , and rotation matrix R . The relative rotation was not passed to the Kalman filter as another inertial-only orientation filter (Madgwick) was used. The drone's previous pose estimate from the Kalman filter was used to convert the relative translation into the world frame before using it in the update step.

1) Predict

The predict stage of the Kalman filter was responsible for taking in the system inputs and determining the new states based on the dynamics of the system. In the case of the drone, the assumption of linear displacement was made due to the quick sampling time of the IMU. The dynamics of the drone flight was based on linear motion laws. Since the input of the predict step is the acceleration of the drone in the drone frame, a couple modifications must be made to the data. Firstly, the linear acceleration was integrated to get velocity. The previous velocity measurement were used to update the position estimate of the drone. Since the time steps were small, it was assumed that the velocity would not change significantly between the time steps. Before processing the acceleration in world frame, the acceleration from the IMU (in IMU/drone frame) needed to be rotated towards the current pose. To convert the acceleration from drone frame to world frame, the pose from the Madgwick filter was fed into a combined rotation matrix. This matrix was applied to

the acceleration to get the acceleration in the world xyz frame.

Once the world frame acceleration was calculated, the position and velocity states of drone were computed, as shown in the equations below:

Velocity:

$$\begin{aligned} v_x[k] &= v_x[k-1] + a_x[k] \cdot \Delta T \\ v_y[k] &= v_y[k-1] + a_y[k] \cdot \Delta T \\ v_z[k] &= v_z[k-1] + a_z[k] \cdot \Delta T \end{aligned}$$

Displacement:

$$\begin{aligned} t_x[k] &= t_x[k-1] + v_x[k] \cdot \Delta T \\ t_y[k] &= t_y[k-1] + v_y[k] \cdot \Delta T \\ t_z[k] &= t_z[k-1] + v_z[k] \cdot \Delta T \end{aligned}$$

The dynamics of the system were derived as a state-space representation. The state vector and inputs are listed below:

$$x[k] = \begin{bmatrix} t_x \\ t_y \\ t_z \\ v_x \\ v_y \\ v_z \end{bmatrix} \quad u[k] = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}$$

The matrices for the dynamics of the system are as follows:

$$A = \begin{bmatrix} 1 & 0 & 0 & \Delta T & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta T & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta T \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \Delta T & 0 & 0 \\ 0 & \Delta T & 0 \\ 0 & 0 & \Delta T \end{bmatrix}$$

$$x[k+1] = Ax[k] + Bu[k]$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$z[k] = Cx[k]$$

The predicted uncertainty of the states were also recomputed at the end of the predict state using the following equation.

$$P_{k|k-1} = AP_{k-1|k-1}A^T + Q$$

2) Update

The measurements of the sensors were incorporated into the state estimate in the update step of the Kalman filter. The predicted state was compared to another measurement (in this case, the VIO output) to fuse the data together. The Kalman gain represented how much of a correction needs to be applied to the states given the prediction and sensor measurements. It was calculated using the error covariance matrix, the measurement noise matrix and the output state matrix. At each update step, the matrix was updated for the new gain. The states were then updated using the Kalman Gain and the outputs of the camera and prediction.

$$K_k = P_{k|k-1}C^T(CP_{k|k-1}C^T + R)^{-1}$$

$$P_{k|k} = (I - K_kC)P_{k|k-1}$$

The estimate was also updated using the states from the predict step and the states of the sensor, in our case being the camera. The estimated measurements are denoted with $\hat{x}$.

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1})$$

3) Implementation

The Kalman filter was implemented in Python by developing a class which encapsulated the predict and update steps. The class is instantiated with initial state, δt , initial covariance, process noise, measurement noise, and the number of states.

The class consists of two functions – update, and predict.

The predict function takes the output of the Madgwick filter - the orientation of the drone in its own frame of reference. These were then converted to world frame using rotation matrices. Using the previous state velocities, the current position of the drone in the world frame was updated. Furthermore, the velocities were also updated using the most recent acceleration data. Finally, the state error covariance matrix, P was updated in the predict step.

The update function took the camera data and the current state of the drone. The camera data was used to calculate the Kalman gain, which was then used to update the estimated state of the drone. The error covariance matrix, P, was also updated in this step.

The linear algebra operations and definition of the matrices were done using the Numpy library. The rotation matrices were defined using the Euler angles from the Madgwick filter.

C. Madgwick Filter Implementation

The Madgwick filter was implemented in Python to provide an orientation pose estimate. The dataset was extracted and processed using Pandas [5] However, due to the nature of the drone's flight and real-world conditions, modifications were required to the original algorithm to ensure the filter was able to provide a stable orientation estimate. The modifications

made to the Madgwick filter are described below.

1) Gyroscope Bias Compensation

Unlike other inertial orientation filters that are capable of measuring bias in the gyroscopic measurements (ex: Multiplicative EKFs), a Madgwick filter is not capable of doing so. A Madgwick filter can implicitly correct for gyroscopic bias with the correction step, but it does so relatively poorly as the bias is not explicitly modelled. As a result, the VIO algorithm calculated the average gyroscopic bias on startup and is subtracted from the gyroscopic measurements within the Madgwick filter implementation before integrating them per Equation 2.

2) High Acceleration Compensation

A key limitation of a Madgwick filter is that it assumes the accelerometer measurements are only affected by gravity, and not by any other forces acting on the drone. This assumption is not valid in all cases, especially when the drone undergoing high acceleration maneuvers. When evaluating the implemented filter's performance in high acceleration scenarios on the tuning dataset, the orientation estimation was found to diverge significantly and increase the RMSE. Given the context of the dataset is an FPV drone where high acceleration maneuvers are common, the correction step size μ was made to be adaptive to the accelerometer measurements. The algorithm for the adaptive step size is shown below, with the parameters x_1 , x_2 , and x_3 being tuned heuristically. This change helped improve the orientation estimation in high acceleration scenarios. The algorithm is as follows:

$$\mu_{\text{out}} = \begin{cases} 0 & \text{if } \|\mathbf{a}\| \leq x_1 \\ \mu \cdot \frac{\|\mathbf{a}\| - x_1}{x_2 - x_1} & \text{if } x_1 < \|\mathbf{a}\| \leq x_2 \\ \mu \cdot \frac{x_3 - \|\mathbf{a}\|}{x_3 - x_2} & \text{if } x_2 < \|\mathbf{a}\| \leq x_3 \\ 0 & \text{if } \|\mathbf{a}\| > x_3 \end{cases}$$

$$\text{where } x_1 = 4.905, \quad x_2 = 9.81, \quad x_3 = 14.715$$

IV. RESULTS AND KEY FINDINGS

A. Drone Pose Estimation

The drone pose estimation from the IMU is required to be accurate for the purposes of the VIO algorithm. Evaluation of the Madgwick filter was performed by comparing the ground-truth pose of the drone to the pose estimated by the Madgwick filter for the validation dataset. The results of the evaluation are plotted using Matplotlib [6] and shown in Figure 4.

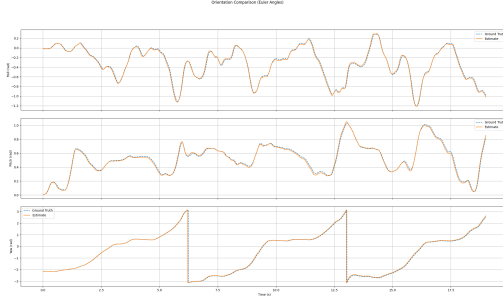


Fig. 4. Madgwick filter results comparing the ground-truth pose of the drone to the estimated pose on the validation dataset.

The results show that the implemented Madgwick filter was able to estimate the pose of the drone with a high degree of accuracy (RMSE (Roll, Pitch, Yaw): [0.024 0.016 0.040] rad) in real-time, making it suitable for use in the VIO algorithm.

B. Kalman Filter Performance

The performance of the Kalman filter was validated by plotting the output of the filter against the ground truth position of the drone in the world frame. The ground truth position was parsed from the dataset, and the Kalman filter was run on the raw sensor data, also parsed from the dataset. Using Matplotlib, both of these results were plotted on a 3D graph, with the x, y and z axes representing the world frame.

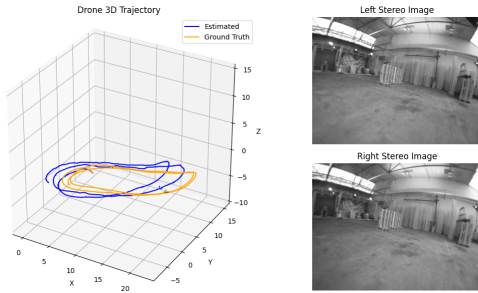


Fig. 5. Kalman filter results on testing set with tracks from both estimated and ground truth performance.

It was observed that the Kalman filter was able to track the ground truth position of the drone on the testing dataset, although there is still drift and deviation that needs to be addressed. Next, the performance of the filter on the validation dataset was plotted and evaluated:

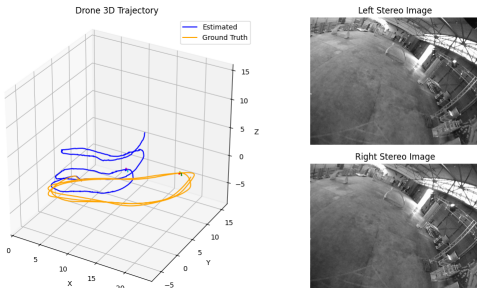


Fig. 6. Kalman filter results on validation set with tracks from both estimated and ground truth performance.

It was observed that the general shape of the movement was tracked by the estimator in the x and y axes. However, there was significant observed drift in the z axis, as seen by the upward corkscrew movement of the drone as per the estimator. Ultimately, the filter was tuned to get an RMSE of [5.02, 3.17, 4.30] for the x, y and z axes respectively.

C. Vision Assessment

The error in the relative transformation output from the vision system was measured across 500 dataset samples. Errors were quantified as the difference between measured and ground-truth transformations in the drone reference frame. Translational errors were computed as vector differences. Rotational errors were represented in Rodriguez vector form. Results are shown in Table I.

Axis	Mean error [m]	Variance [m]	RMS error [m]
x	-0.0908	0.0102	0.1358
y	0.0123	0.0083	0.0918
z	-0.0302	0.0050	0.0767
Axis	Mean error [rad]	Variance [rad]	RMS error [rad]
θ_x	0.0000	0.0005	0.0229
θ_y	0.0006	0.0001	0.0121
θ_z	0.0026	0.0005	0.0230

TABLE I
ERROR STATISTICS FOR POSITION AND ROTATION

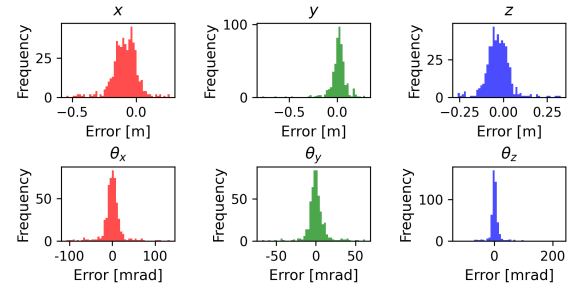


Fig. 7. Distribution of error components in relative vision odometry output. Measured across 500 samples of the validation dataset.

D. Limitations

From the results of the filter, it is apparent that, while performing better than a pure dead-reckoning approach, the Kalman filter exhibits drift in its position estimation. Figure 8 shows the results of an inertial dead-reckoning approach on the training dataset, demonstrating the improvement offered with the vision-based fusion. The RMSE of the dead-reckoning approach is 7.01, 10.08, and 0.91 meters for x, y, and z respectively.

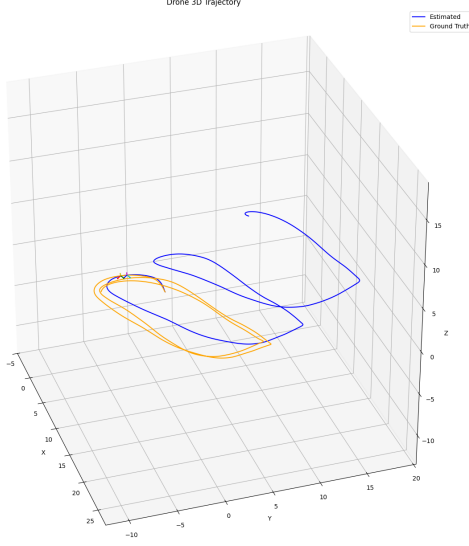


Fig. 8. Dead-reckoning results comparing the ground-truth pose of the drone to the estimated pose on the training dataset.

The above figure shows the significant amount of drift that results from a pure dead-reckoning approach in the X/Y plane. The Kalman filter is able to reduce this drift significantly, but it is still present. While the orientation estimation is quite accurate, the Madgwick filter has no way of correcting for yaw drift in the absence of another sensor (ex: magnetometer). As a result, a poor yaw estimate will cause the position estimation to drift, especially as the motion model directly integrates acceleration measurements in the estimated body frame.

E. Discussion

As a result of the above limitations, the VIO fused algorithm performed better than a dead-reckoning approach. Strengths of the algorithm include:

- Elimination of drift in the X/Y position with camera correction input
- Accurate orientation pose estimation in the world frame with the Madgwick filter

Clear limitations of the algorithm include:

- Drift in the Z axis introduced by the fusion of the visual odometry algorithm - while the cause of this is unknown, a possible explanation is that the camera is not able to accurately measure the Z axis due to the nature of the stereo camera setup during high speed flight or turning motions. As a result, the VIO algorithm may falsely integrate poor Z axis measurements, causing drift. This may be mitigated with further tuning of the VIO algorithm or by adding another sensor (ex: barometer) to correct for Z axis drift.
- Yaw drift from the Madgwick filter as run time increases - this may be correctable by employing a magnetometer or with another type of filter that is able to incorporate yaw correction from visual odometry.
- Lack of a robust motion model for the Kalman filter - the current model is a simple constant velocity model,

which does not account for acceleration, deceleration, or turning motions. This may be improved with a more complex motion model that is able to account for these factors using interactive multiple model (IMM) filtering or a more complex Kalman filter model.

V. CONCLUSION

This report covered the development of a state estimation system for drone pose and position using measurements from an IMU and a camera. The overall state estimator was based on a Kalman filter. The predict step employed a Madgwick filter to obtain a filtered orientation of the drone, which was then used to rotate the IMU acceleration data into the world frame. The camera sensors were used to perform visual-inertial odometry and to estimate the drone's position. The camera's position estimate served as the measurement for updating the Kalman filter. The Kalman filter was tuned using process and measurement noise covariance matrices, which were iteratively refined to achieve the best fit. The final RMSE of the Kalman filter was [5.02, 3.17, 4.30] meters in the x, y, and z axes respectively, when tested on a validation dataset.

Further recommendations to improve the overall VIO and Kalman filter performance include investigating position drift in the VIO, which causes the overall state estimate to diverge over time. Another major improvement would involve using profiling tools to identify bottlenecks in the code and algorithm. This would help optimize performance and enable more reliable real-time operation. Such improvements are important for deployment on a drone, where low-latency and real-time performance are critical.

APPENDIX A

DIAGRAMS FOR VISION ODOMETRY UNDISTORTION, TRIANGULATION, AND POINT CLOUD MATCHING

Figure 9 shows the process of preparing matched stereo image features for triangulation. The intrinsic camera matrices provided in the dataset were used for undistortion and inverse projection. Triangulation also used the relative transformation matrix between the two cameras in the stereo pair, and was done with the OpenCV triangulate function [4].

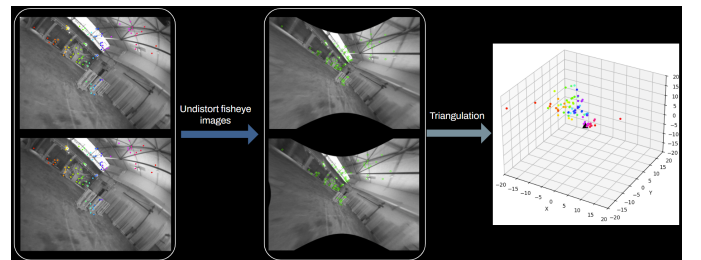


Fig. 9. Process for taking matched features, undistorting them, and creating a point cloud with triangulation.

Figure 10 shows how point clouds from consecutive stereo frames are aligned. A relative transformation is computed to

estimate motion between frames based on corresponding 3D points.

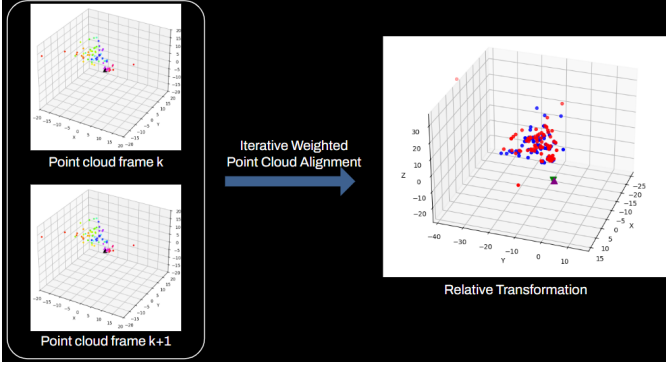


Fig. 10. Process for taking point clouds from two consecutive stereo frames and finding a relative transformation.

REFERENCES

- [1] J. Delmerico, T. Cieslewski, H. Rebecq, M. Faessler, and D. Scaramuzza, “Are we ready for autonomous drone racing? the UZH-FPV drone racing dataset,” in *IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019.
- [2] e. a. Bayard David S., “Vision-based navigation for the nasa mars helicopter,” in *AIAA Scitech 2019 Forum*, 2019.
- [3] *Madgwick filter — ahrs documentation*, <https://ahrs.readthedocs.io/en/latest/filters/madgwick.html>, Accessed: 2025-04-07.
- [4] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [5] W. McKinney, “Data structures for statistical computing in python,” in *Proceedings of the 9th Python in Science Conference*, S. van der Walt and J. Millman, Eds., 2010, pp. 51–56.
- [6] J. D. Hunter, “Matplotlib: A 2d graphics environment,” *Computing in science & engineering*, vol. 9, no. 3, p. 90, 2007.